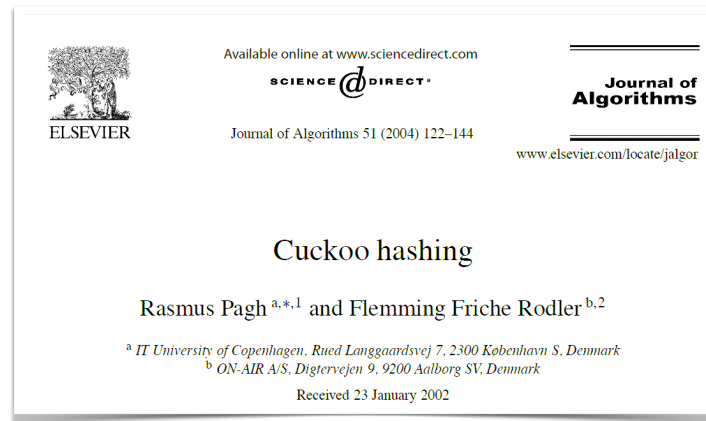


Algorithm Design & Analysis

Course Project

Cuckoo Hashing



Second Paper

Robin Hood Hashing



Laying its eggs in the nests of
other birds

Leaving the task of eggs
incubating host birds.



When cuckoo chick hatches,
it pushes the host's eggs or chicks
out of the nest

Monopolize
food and care



Cuckoo bird
Life Cycle

About the paper

Paper information:

- Title: Cuckoo Hashing
- Highly cited paper
- Authors: Rasmus Pagh & Flemming Friche Rodler
- Journal: Journal of Algorithms, V 51, Issue 2, P 122-144
- Publication Date: 2004
- DOI: 10.1016/j.jalgor.2003.12.002

Cuckoo hashing

[R Pagh](#), FF Rodler - Journal of Algorithms, 2004 - Elsevier

... is a new **hashing** scheme called **CUCKOO HASHING**, which ... to an asymptotically small **hash** function description) there are just ... Using weaker **hash** functions than those required for our ...

☆ Save  Cite Cited by 1566 Related articles All 12 versions

Cuckoo Hashing Variants:

- d-ary Cuckoo Hashing
- Cuckoo Hashing with Stash
- Asymmetric Cuckoo Hashing
- Cuckoo Hashing with Multiple Buckets
- Partitioned Cuckoo Hashing
- Cuckoo Filter

About the paper

Cuckoo Hashing is a technique for resolving collisions in hash tables that produces a dictionary data structure:

- Inspired by natural behaviour of Cuckoo Bird.
- Search and lookup $O(1)$ - 2 probes 🏆
- Delete $O(1)$ - 2 probes
- Insertion $O(1)$ amortized

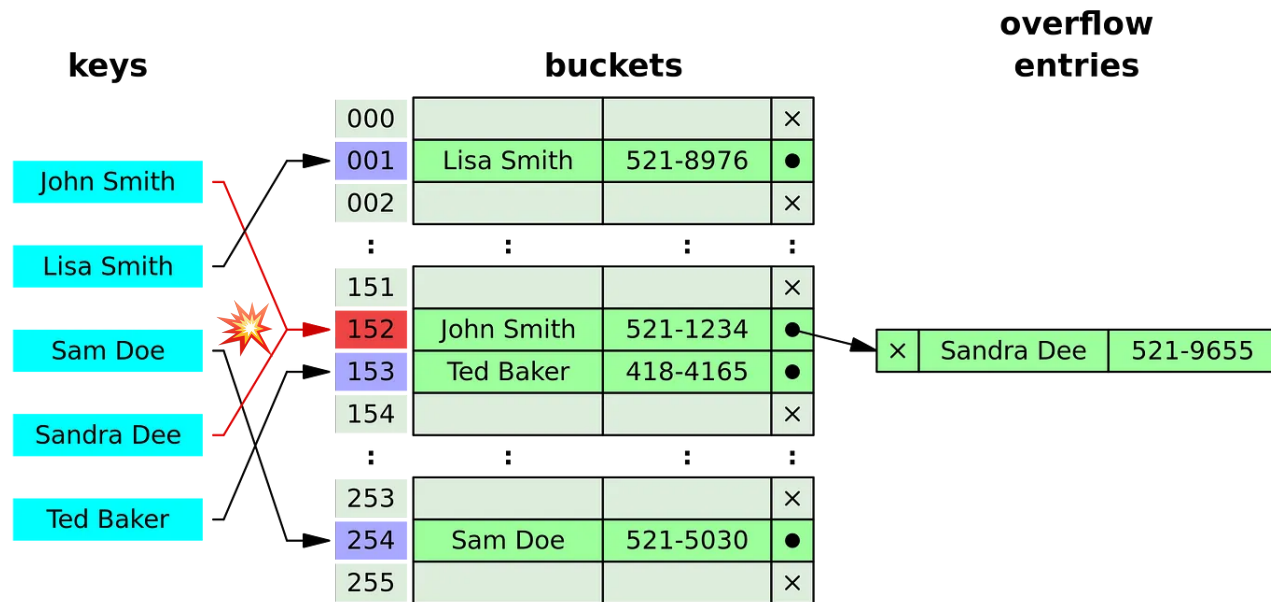
Problem statements:

- However, in the absence of perfect hash functions mapping keys injectively to locations in a set of buckets, collisions where two distinct keys map to a single bucket location to be resolved.

Broader view of hashing schemes | Chained hashing

Chained hashing:

- linked list is used as a data structure to store keys & buckets locations.
- Collisions are resolved by lengthening the list structures in buckets with collisions.



n : number of keys

m : number of buckets

load factor (α): $\frac{n}{m}$

Average case:

- Lookup: $O(1 + \alpha)$
- Insertion: $O(1)$
- Deletion: $O(1 + \alpha)$

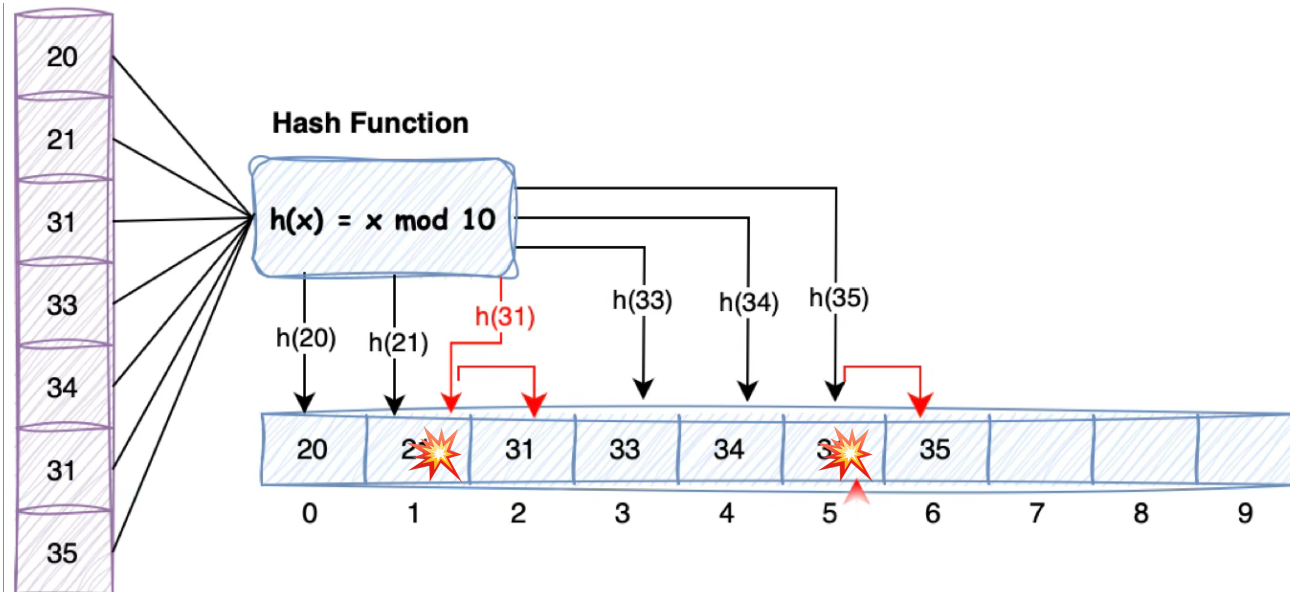
Worst case:

- Lookup: $O(n)$
- Insertion: $O(n)$
- Deletion: $O(n)$

Broader view of hashing schemes | Linear Probing

Linear probing:

- keys are hashed to form a bucket index.
- On insertion, if the bucket not available, scan for a free bucket.
- During lookup, find a starting bucket, if not existed, go for consecutive buckets. Stop when:
 - Key is found
 - Empty bucket is reached.



Average case:

- Lookup: $O(1)$
- Insertion: $O(1)$
- Deletion: $O(1)$

Worst case:

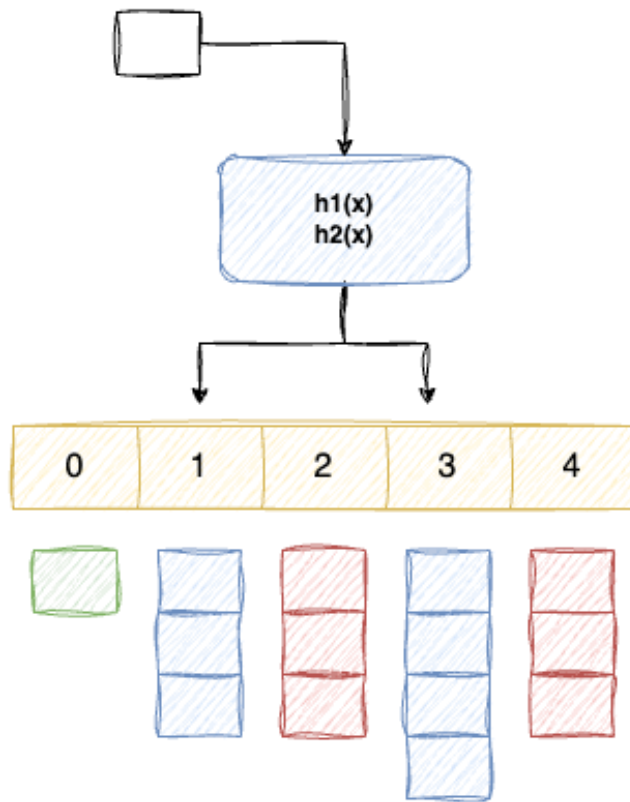
- Lookup: $O(n)$
- Insertion: $O(n)$
- Deletion: $O(n)$

Broader view of hashing schemes | Two-way Chaining

Two way chaining:

- Compute two hashes.
- Compare chain lengths
- Insert in shorter chain.
- Lookup checks both chains

The shorter of two buckets makes very long chains much less likely.



Average case

- Lookup: $O(1)$
- Insertion: $O(1)$
- Deletion: $O(1)$

Worst case:

- Lookup: $O(\log \log n)$
- Insertion: $O(1)$
- Deletion: $O(\log \log n)$

Note: it can be cache friendly version: each chain is a small fixed size array. If full, then table is rebuilt using new hash functions.

Cuckoo Hashing

- **Motivation:**

- Many hash tables are fast on average, but a bad collision pattern can make lookup longer.

- **Gola:**

- Very fast lookup
- Fast updates

- **Core Idea:**

- Two hash tables T_1, T_2 with size m each and two
- hash functions h_1 for T_1 and h_2 for T_2 .
- Can use for instance $h_1(k) = k \bmod m$ and $h_2(k) = \lceil k/m \rceil \bmod m$
- Every element $k \in U$ can be only in two positions: at $h_1(k)$ in T_1 or at $h_2(k)$ in T_2 .

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	20

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	20

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	50
7	
8	
9	20

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
★ 53	9	4

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	
2	
3	
4	
5	
6	50
7	
8	
9	20

T_1

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

T_2

C: 0



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

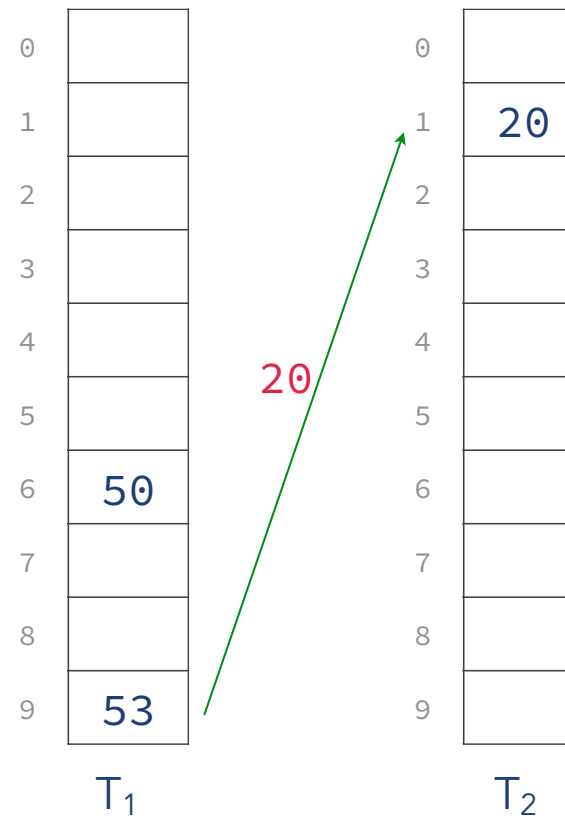
If $C \geq \max$ loop, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
★ 53	9	4

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



C: 1

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

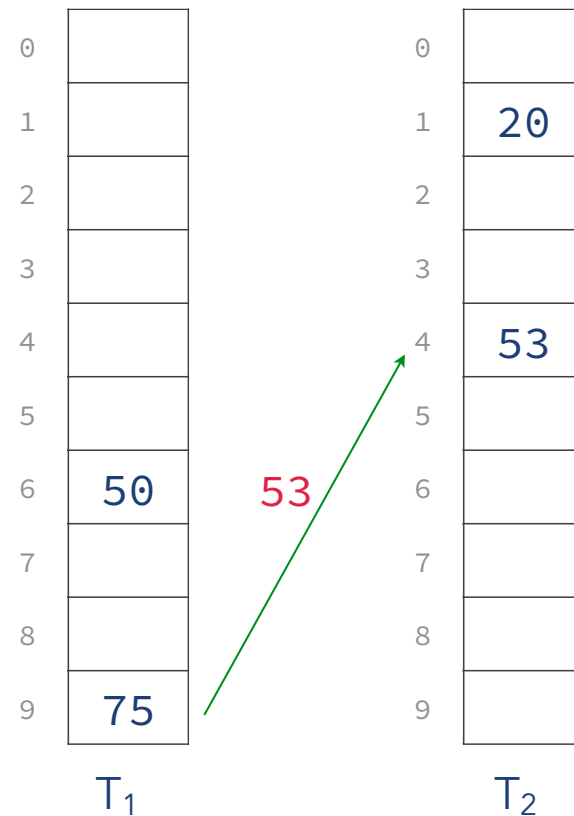
If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✱ 53	9	4
✱ 75	9	6

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max$ loop, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✶ 53	9	4
✶ 75	9	6
100	1	9

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	100
2	
3	
4	
5	
6	50
7	
8	
9	75

T_1

0	
1	20
2	
3	
4	53
5	
6	
7	
8	
9	

T_2

C: 0

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

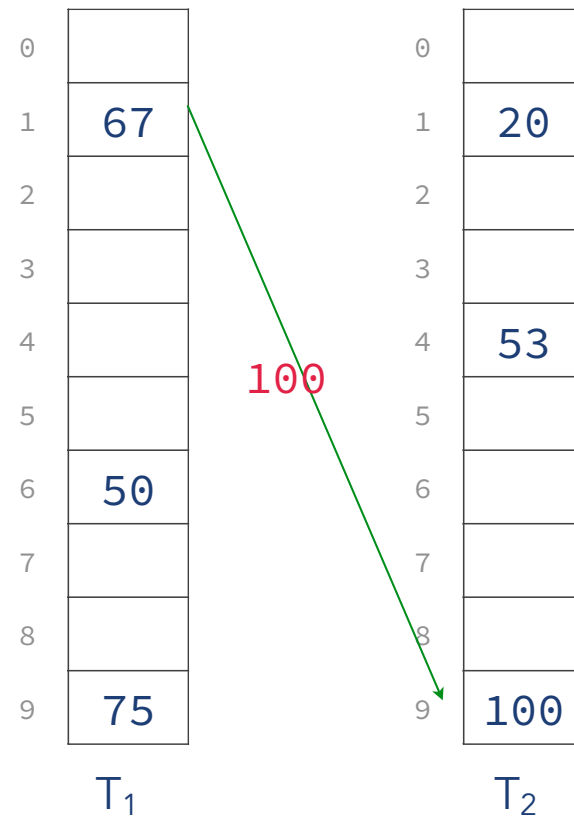
If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✱ 53	9	4
✱ 75	9	6
100	1	9
✱ 67	1	6

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



C: 1

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

$C=0$, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✱ 53	9	4
✱ 75	9	6
100	1	9
✱ 67	1	6
✱ 105	6	9

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

0	
1	67
2	
3	
4	
5	
6	105
7	
8	
9	75

T_1

0	
1	20
2	
3	
4	53
5	
6	
7	
8	
9	100

T_2

C: 1



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

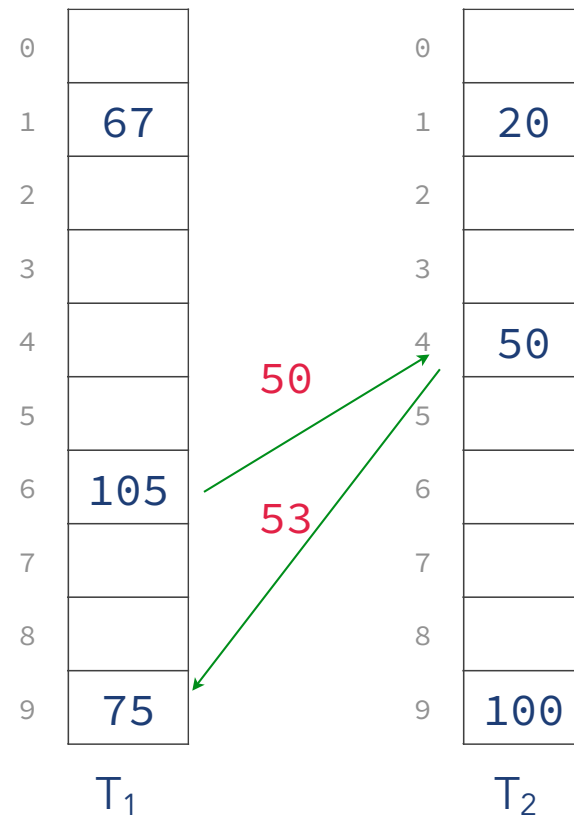
If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✱ 53	9	4
✱ 75	9	6
100	1	9
✱ 67	1	6
✱ 105	6	9

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

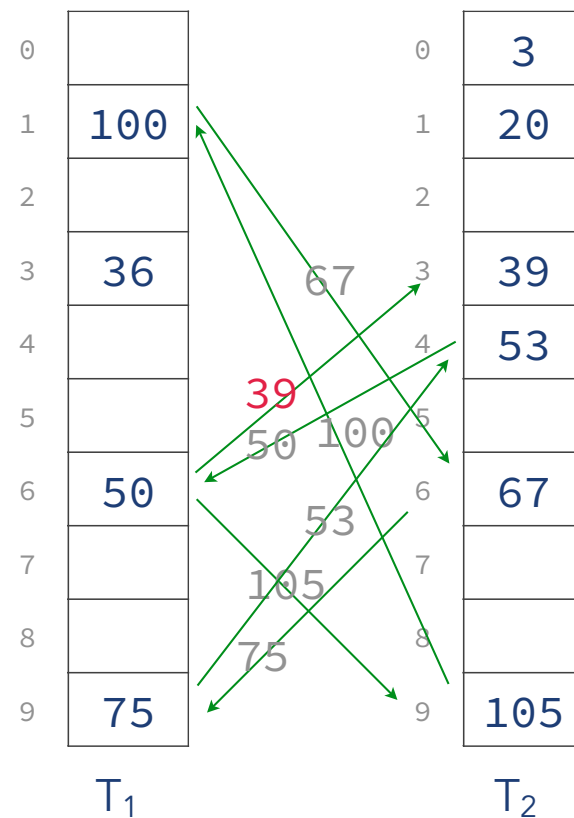
If $C \geq \max$ loop, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✱ 53	9	4
✱ 75	9	6
100	1	9
✱ 67	1	6
✱ 105	6	9
3	3	0
✱ 36	3	3
✱ 39	6	3

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



C: 7

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

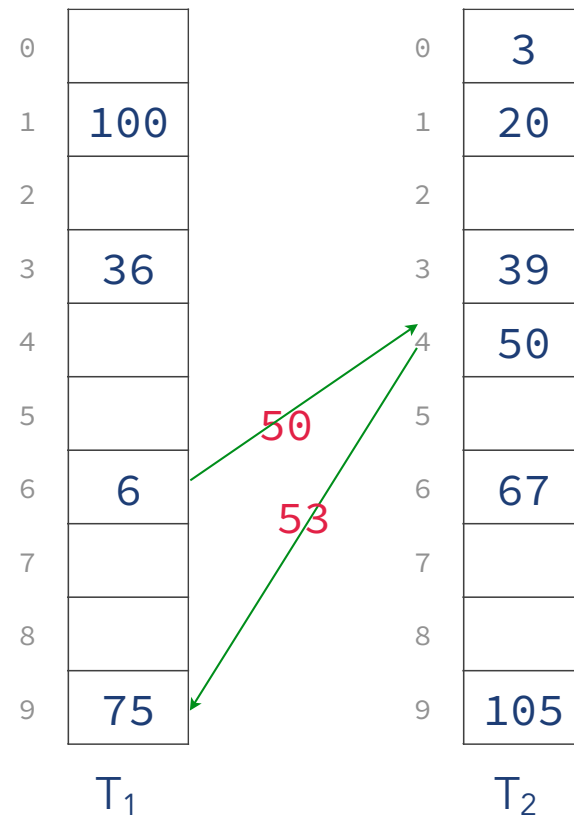
If $C \geq \max$ loop, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

	k	h_1	h_2
	20	9	1
	50	6	4
💣	53	9	4
💣	75	9	6
	100	1	9
💣	67	1	6
💣	105	6	9
	3	3	0
💣	36	3	3
💣	39	6	3
💣	6	6	0

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

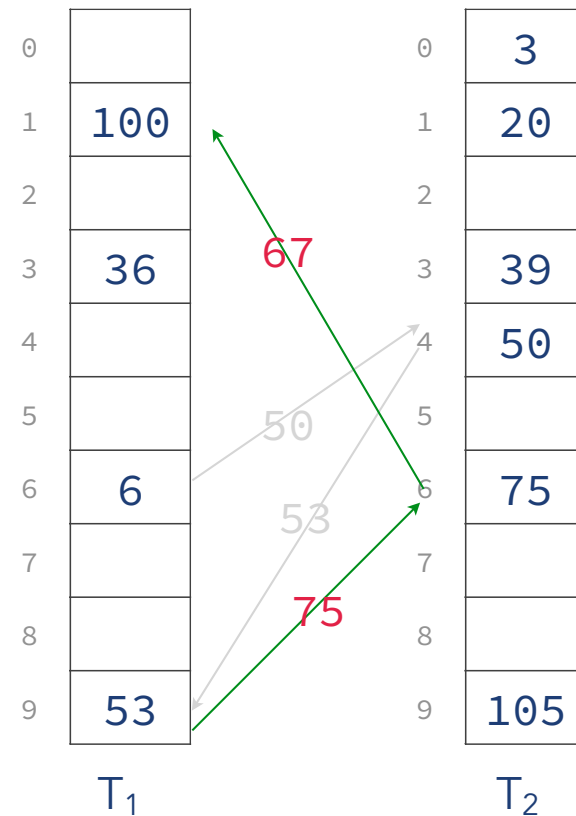
If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys:(20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

	k	h_1	h_2
	20	9	1
	50	6	4
💣	53	9	4
💣	75	9	6
	100	1	9
💣	67	1	6
💣	105	6	9
	3	3	0
💣	36	3	3
💣	39	6	3
💣	6	6	0

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

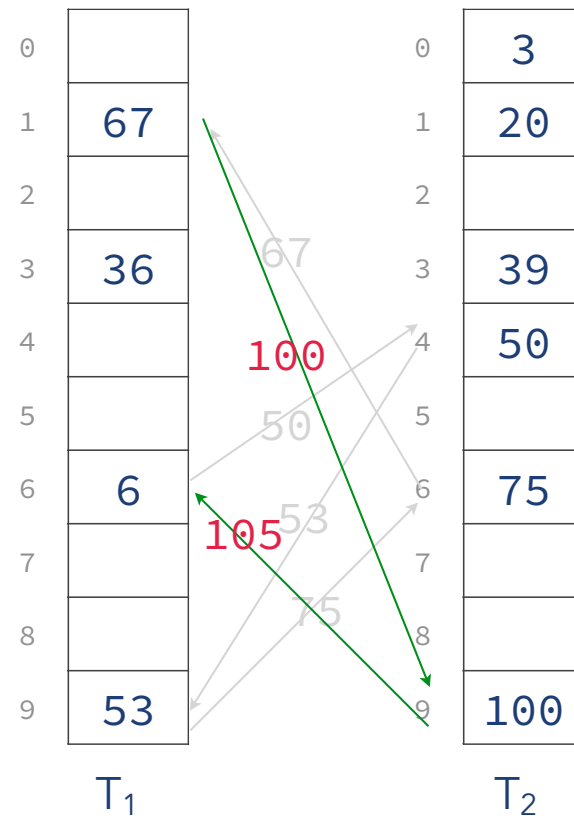
If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

	k	h_1	h_2
	20	9	1
	50	6	4
💣	53	9	4
💣	75	9	6
	100	1	9
💣	67	1	6
💣	105	6	9
	3	3	0
💣	36	3	3
💣	39	6	3
💣	6	6	0

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



C: 6

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

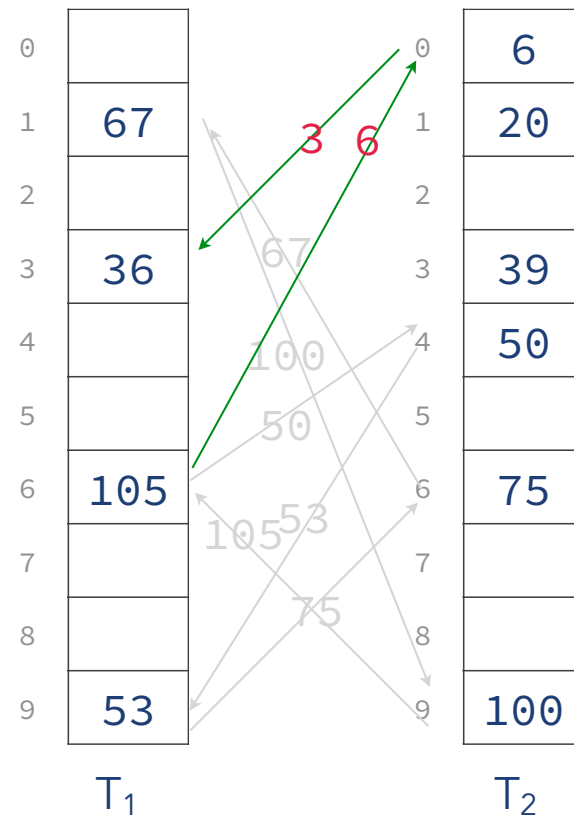
If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

	k	h_1	h_2
	20	9	1
	50	6	4
💣	53	9	4
💣	75	9	6
	100	1	9
💣	67	1	6
💣	105	6	9
	3	3	0
💣	36	3	3
💣	39	6	3
💣	6	6	0

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$



C: 8

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max \text{ loop}$, rehash.

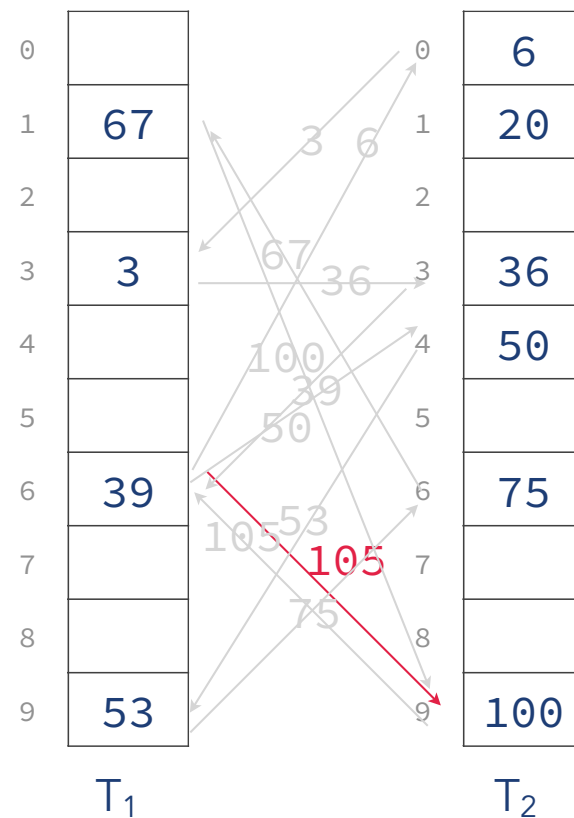
Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	9	1
50	6	4
✪ 53	9	4
✪ 75	9	6
100	1	9
✪ 67	1	6
✪ 105	6	9
3	3	0
✪ 36	3	3
✪ 39	6	3
💣 6	6	0

$\max \text{ loop} = 11$

$h_1(k) = k \bmod 11$

$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$



C: 11

Cuckoo Hashing | Insert

Initiate two tables A & B. Size of each m.

While you have keys:

C=0, Compute the two possible locations for x.

Place x if one location is empty.

Otherwise, kick out the existing key and move it to its other location, update counter.

Repeat until an empty slot is found.

If $C \geq \max \text{ loop}$, rehash.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

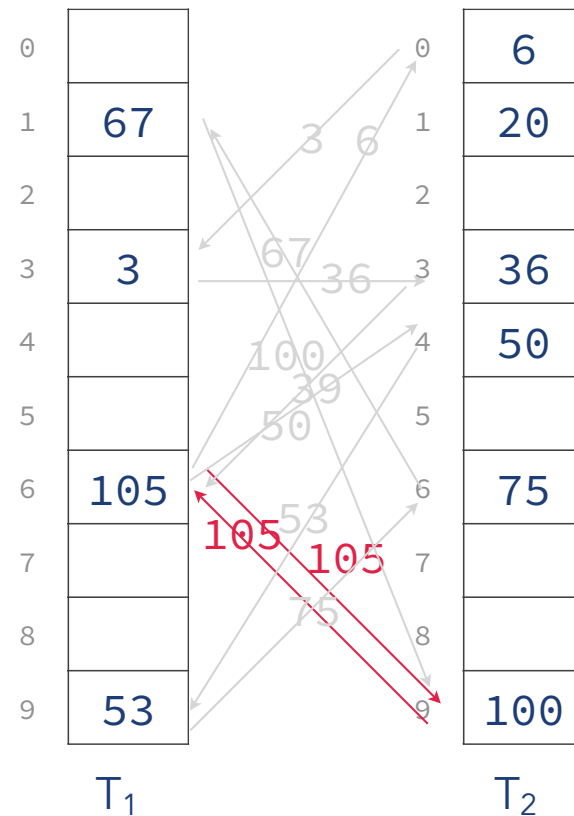
k	h_1	h_2
20	9	1
50	6	4
✪ 53	9	4
✪ 75	9	6
100	1	9
✪ 67	1	6
✪ 105	6	9
3	3	0
✪ 36	3	3
✪ 39	6	3
💣 6	6	0

$\max \text{ loop} = 11$

$$h_1(k) = k \bmod 11$$

$$h_2(k) = \left\lfloor \frac{k}{11} \right\rfloor \bmod 11$$

Loop .. However, CH
does not check loops



C: 11

Cuckoo Hashing | Rehashing

Create two new hash tables T'_1 , T'_2 .

Choose new random hash functions h_1 and h_2 .

Collect all keys from T_1 and old T_2 .

Insert every key again using the new h_1 and h_2 .

If successful, replace T_1 , T_2 with T'_1 , T'_2 .

If it fails again, try another rehash or grow the tables.

Example: Insert the set of keys: (20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 6)

k	h_1	h_2
20	10	4
50	3	10
53	4	10
75	4	4
100	6	9
67	0	2
105	8	10
3	1	0
36	7	7
39	8	7
6	3	1

$\max \text{ loop} = 11$

$$h'_1(k) = \left\lfloor \frac{k}{2} \right\rfloor \bmod 11$$

$$h'_2(k) = \left\lfloor \frac{k}{5} \right\rfloor \bmod 11$$

0		0	6	0	67	0	
1	67	1	20	1	3	1	
2		2		2		2	
3	3	3	36	3	6	3	
4		4	50	4	53	4	75
5		5		5		5	
6	105	6	75	6	100	6	
7		7		7	36	7	39
8		8		8	105	8	
9	53	9	100	9		9	
				10	20	10	50
	T_1		T_2		T'_1		T'_2

Cuckoo Hashing | Insert Cost

Insert:

- Two possible positions: $h_1(key)$ or $h_2(key)$
- Insertion tries one position, then num of displacements $O(numDisplacements)$
- Expected insertion cost: $O(1)$
- Expected amortized insertion cost: $O(1)$

Worst case insertion:

- Worst case happens when insertion enters a cycle or long chain of displacements
- Nestless key can find position at number of displacements $O(numDisplacements)$
- Also a cycle and / or reaching number of displacements triggers rehashing: $O(n)$
- However $O(n)$ is unlikely, but if it happens, table sizes are doubled.

Cuckoo Hashing | Insert Cost + Implementation

- Cost: $O(\text{numDisplacements} + n)$

```
procedure insert(x)
  if lookup(x) then return
  loop MaxLoop times
     $x \leftrightarrow T_1[h_1(x)]$ 
    if  $x = \perp$  then return
     $x \leftrightarrow T_2[h_2(x)]$ 
    if  $x = \perp$  then return
  end loop
  rehash(); insert(x)
end
```

$O(\text{numDisplacements})$

```
void CuckooHashing::insert(int key) {
  ++insertAttempts;
  const uint32_t k = normalizeKey(key);
  lastInsertCellAccesses = 0;
  const int i1 = hash10f(k);
  ++lastInsertCellAccesses;
  if (T1[i1] == k) {
    ++duplicateInsertions;
    insertCellAccessSum += lastInsertCellAccesses;
    ++sampledInsertCount;
    return;
  }

  const int i2 = hash20f(k);
  ++lastInsertCellAccesses;
  if (T2[i2] == k) {
    ++duplicateInsertions;
    insertCellAccessSum += lastInsertCellAccesses;
    ++sampledInsertCount;
    return;
  }

  if (placeKeyInLoop(k)) {
    ++elementCount;
    ++successfulInsertions;
    ++insertionsSinceRehash;
    insertCellAccessSum += lastInsertCellAccesses;
    ++sampledInsertCount;
    return;
  }
}
```

placeKeyInLoop implementation: next slide

Cuckoo Hashing | Insert Cost + Implementation

- Cost: $O(\text{numDisplacements} + n)$

$O(\text{numDisplacements})$ 

```
procedure insert(x)
  if lookup(x) then return
  loop MaxLoop times
     $x \leftrightarrow T_1[h_1(x)]$ 
    if  $x = \perp$  then return
     $x \leftrightarrow T_2[h_2(x)]$ 
    if  $x = \perp$  then return
  end loop
  rehash(); insert(x)
end
```

Three cases to exist from loop:

- Return from T1 - free slot found
- Return from T2 - free slot found
- $\text{step} \geq \text{maxloop}$

```
bool CuckooHashing::placeKeyInLoop(uint32_t key) {
  uint32_t newKey = key;
  bool placeInT1 = true;

  for (int step = 0; step < maxLoop; ++step) {
    if (step > 0) {
      ++lastInsertCellAccesses;
    }
    if (placeInT1) {
      const int idx = hash10f(newKey);
      T1 if (T1[idx] == EMPTY_SLOT) {
        T1[idx] = newKey;
        return true;
      }
      ++displacementCount;
      swap(newKey, T1[idx]);
    } else {
      const int idx = hash20f(newKey);
      T2 if (T2[idx] == EMPTY_SLOT) {
        T2[idx] = newKey;
        return true;
      }
      ++displacementCount;
      swap(newKey, T2[idx]);
    }
    placeInT1 = !placeInT1;
  }
  return false;
}
```

Cuckoo Hashing | Rehashing + Implementation

- Cost: $O(\text{numDisplacements} + n)$

```
procedure insert(x)
  if lookup(x) then return
  loop MaxLoop times
     $x \leftrightarrow T_1[h_1(x)]$ 
    if  $x = \perp$  then return
     $x \leftrightarrow T_2[h_2(x)]$ 
    if  $x = \perp$  then return
  end loop
  rehash(); insert(x)
end
```

$O(n)$ →

```
if (placeKeyInLoop(k)) {
  ++elementCount;
  ++successfulInsertions;
  ++insertionsSinceRehash;
  insertCellAccessSum += lastInsertCellAccesses;
  ++sampledInsertCount;
  return;}

```

```
if (!forcedRehashWithPendingKey(k)) {
  ++failedInsertions; }

```

$O(n)$ →

$O(n)$ →

$O(n)$ →

```
const long long r = static_cast<long long>(tableSize);
if (insertionsSinceRehash >= r * r) {
  const vector<uint32_t> keys = collectKeys();
  if (!rebuildTablesWithKeys(keys)) {
    resizeTables(tableSize * 2);
    if (!rebuildTablesWithKeys(keys)) { // Extremely unlikely
      throw std::runtime_error("failed after grow");
    }
  }
}
```

Rehashing is associated with insertion operation:

- Number of displacement (steps) $\geq \text{maxLoop}$
- Total insertions since last rehash $\geq r^2$

Table size is doubling (extremely unlikely):

- When rehashing is failed

Cuckoo Hashing | Lookup + Implementation

- Cost: $O(1)$

```
bool CuckooHashing::search(int key) {  
    if (key <= 0) {  
        return false;  
    }  
    const uint32_t k = static_cast<uint32_t>(key);  
    return T1[hash10f(k)] == k || T2[hash20f(k)] == k;  
}
```


Cuckoo Hashing | Delete + Implementation

- Cost: $O(1)$

```
void CuckooHashing::remove(int key) {  
    if (key <= 0) {  
        return;  
    }  
    const uint32_t k = static_cast<uint32_t>(key);  
  
    const int i1 = hash10f(k);  
    if (T1[i1] == k) {  
        T1[i1] = EMPTY_SLOT;  
        --elementCount;  
        return;  
    }  
  
    const int i2 = hash20f(k);  
    if (T2[i2] == k) {  
        T2[i2] = EMPTY_SLOT;  
        --elementCount;  
        return;  
    }  
}
```

Cuckoo Hashing | Table size + Implementation

Practice:

- the size that should be power of two is:

$$|T_1| = |T_2| = 2^q$$

- where q is tableBits.

keys (n)	Desired load	Total slots	Size of each table
1024	(1/4)	4096	2048
2^{22}	(1/4)	2^{24}	2^{23}

- Golden parameters:
 - Load factor below $\alpha < 0.5$. Size of each table bigger than # keys.
 - Table size power of two.
 - maxLoop: $\lceil 3 \log_{1+\epsilon} r \rceil$ bound such as 64 or 128.

Cuckoo Hashing | Hashing

Theoretical :

- The paper assumes h_1 and h_2 come from a sufficiently universal hash family.

$$\left\{ x \mapsto \left(\left(\sum_{l=0}^{k-1} a_l x^l \right) \bmod p \right) \bmod r \mid 0 \leq a_0, a_1, \dots, a_{k-1} < p \right\}$$

- x : the key to be hashed
- $a_0, a_1, \dots, a_{k-1}$: random coefficients of the polynomial
- k : degree limit / number of coefficients
- p : prime number used for modular reduction
- r : hash table size or output range
- $\bmod p$: keeps the polynomial value within a finite field
- $\bmod r$: maps the result to a table index from 0 to $r - 1$

Universal hash family is used for proof purposes (i.e. high random functions)

Cuckoo Hashing | Hashing + Implementation

Practice:

- The implementation uses fast multiplicative hashing:

$$h_a(x) = \left\lfloor \frac{ax \bmod 2^w}{2^{w-q}} \right\rfloor \equiv h_a(x) = (ax \bmod 2^w) \gg (w - q)$$

- x : input key
- a : random odd large multiplier
- w : word size, e.g. 32 or 64 bits
- q : number of output bits

In practice, CH uses fast hashing function for performance purposes.

MULTIPLIERS_PER_HASH = 3

Example:

$$w = 6, q = 2, a = 3, x = 7$$

$$h_a(x) = \left\lfloor \frac{21}{2^4} \right\rfloor = 1$$

```
uint32_t CuckooHashing::multiplicativeHash( uint32_t key,
      const array<uint64_t, MULTIPLIERS_PER_HASH>& multipliers )
{
    const unsigned shift = 64u - static_cast<unsigned>(tableBits);
    const uint64_t k = static_cast<uint64_t>(key);
    uint64_t accumulator = 0;
    for (std::size_t i = 0; i < MULTIPLIERS_PER_HASH; ++i) {
        accumulator ^= (multipliers[i] * k) >> shift;
    }
    return static_cast<uint32_t>(accumulator);
}
```

$$010101 \gg 4 = 01$$

$$h_a(x) = 1$$

`multiplicativeHash` is called by
`hash10f(k)` and `hash20f(k)`

Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

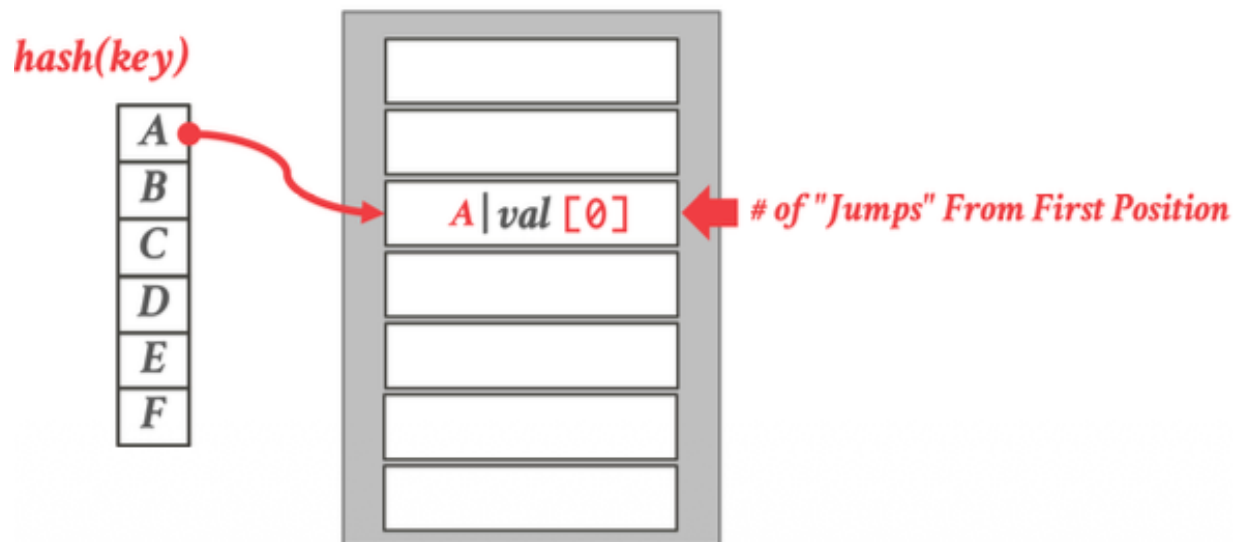
Key Idea: when a key is already present during an insertion that is closer to its “base” location than the new key, it is displaced to make room for new key.

Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

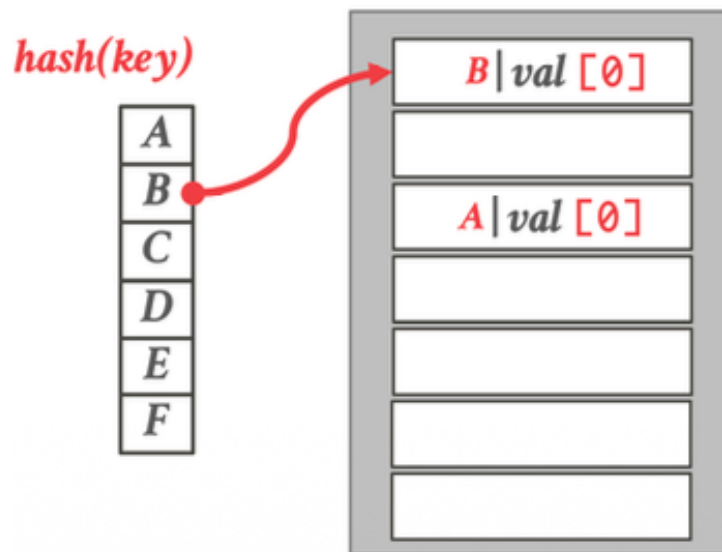


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

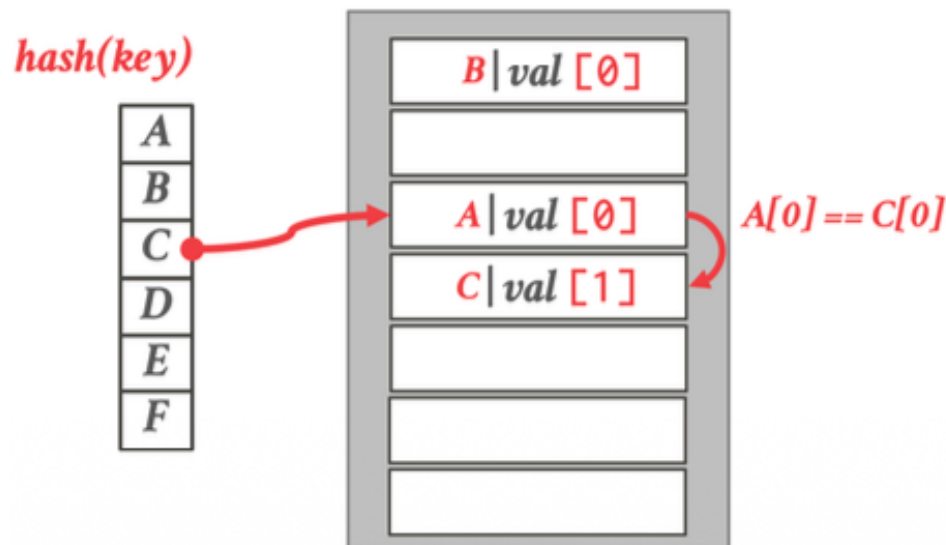


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

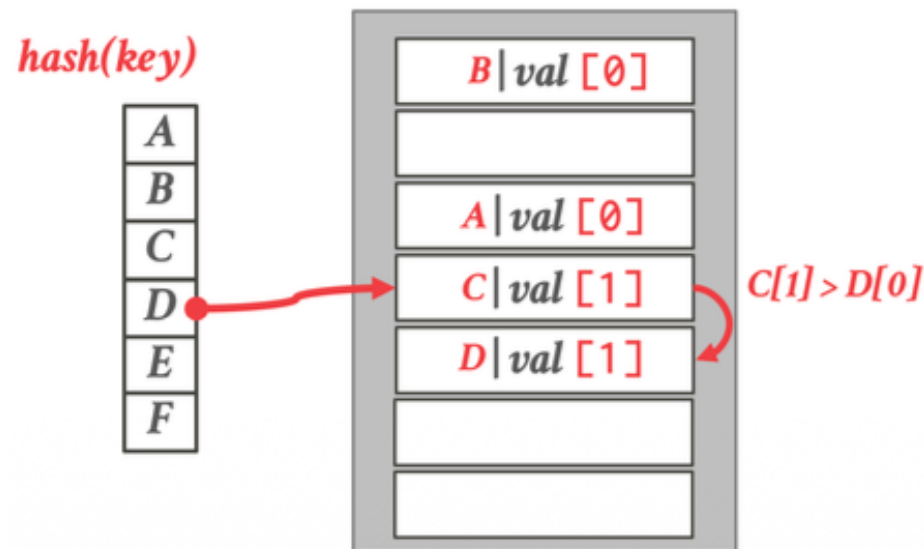


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

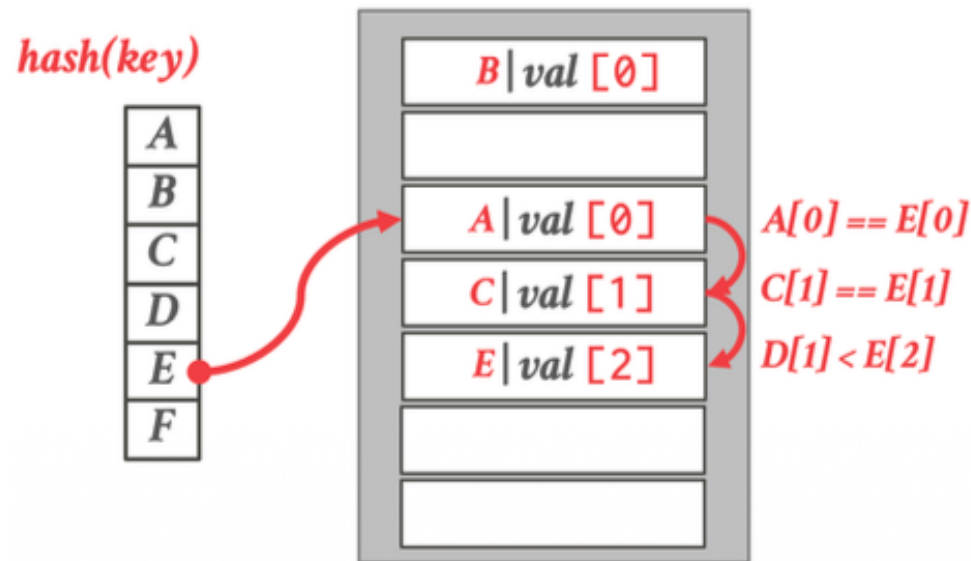


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

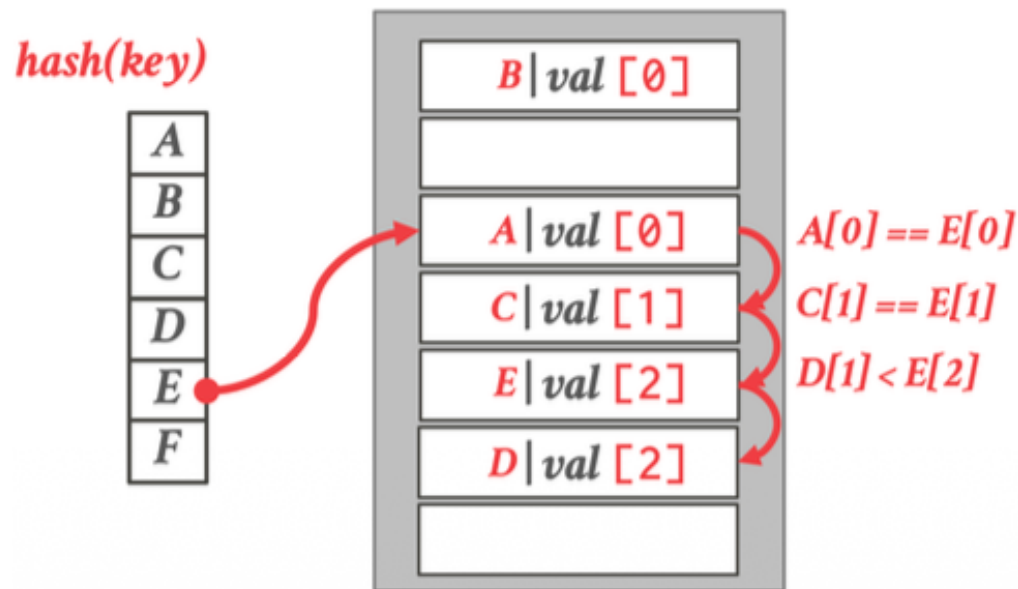


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

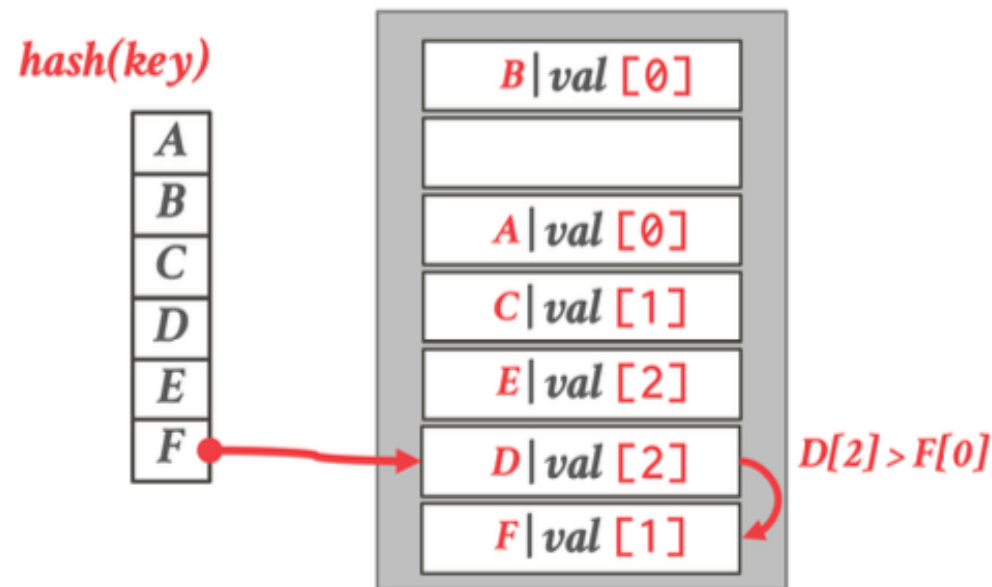


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Insert example:

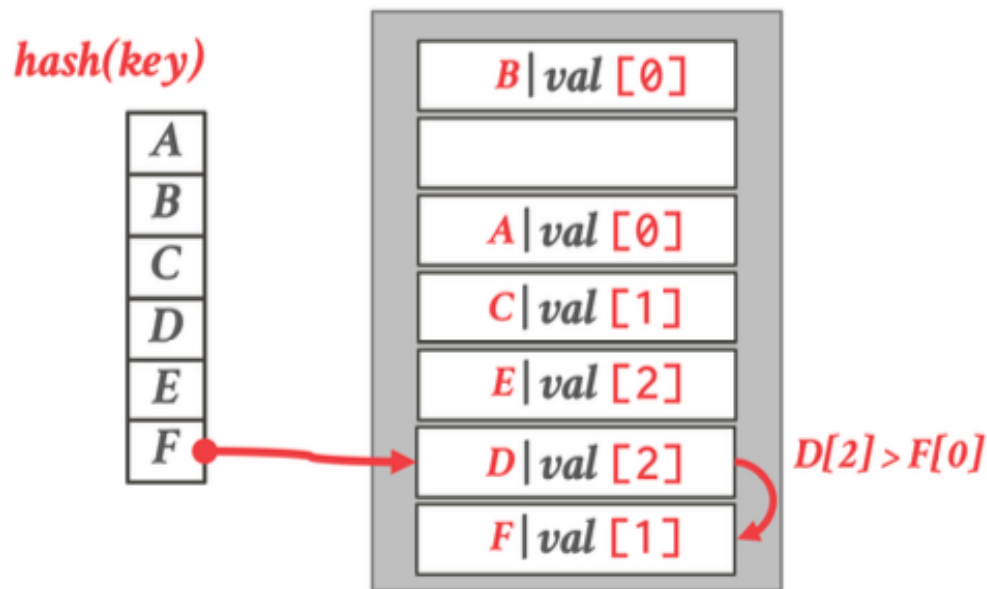


Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Example:



Lookup:

$i = h(X)$, Start checking from index i .
Move forward one cell at a time.

Stop in three cases:

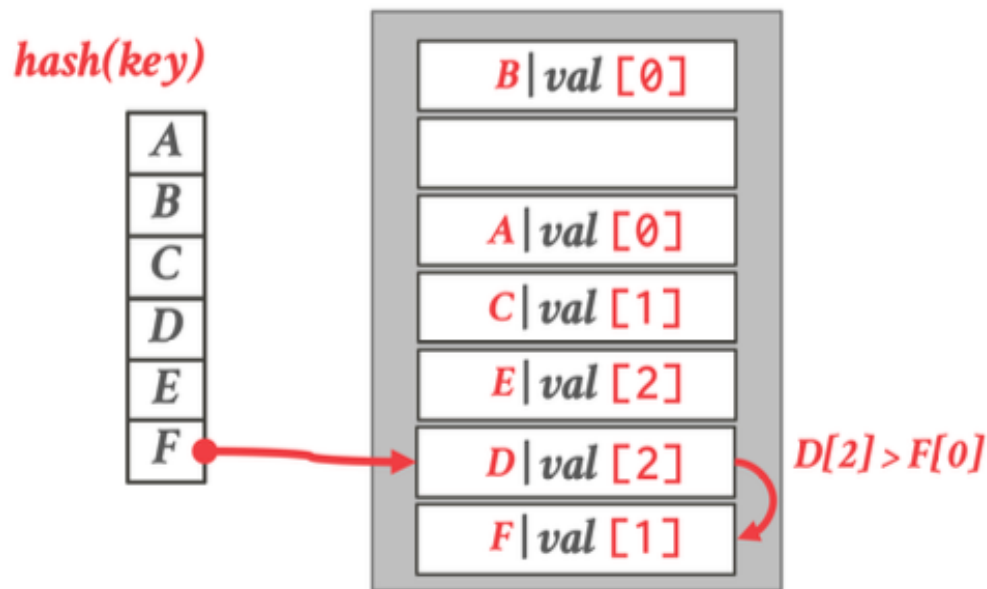
1. Key found
2. Empty cell found
3. Current key has $\leq$ last distance

Second Paper | Robin Hood

Variant of linear probe hashing

- It steals slots from rich keys and give them to poor keys.
- Each key tracks the number of positions they are from where its optimal position in the table.
- On insert, a key takes the slot of another key if the first key is farther away from its optimal position than the second key.

Example:



Delete:

Search for X

Delete X

Backward shift, stop in two cases:

1. an empty cell
2. a key with distance 0

Cuckoo vs Robin Hood Hashing

Operation	Cuckoo Hashing	Robin Hood Hashing
lookup average	$O(1)$	$O(1)$
lookup worst case	$O(1)$	$O(n)$
Insert	$O(1)$ amortized	$O(1)$ average
Insert worst case	$O(n)$	$O(n)$
Delete	$O(1)$	$O(1)$ average
Delete worst case	$O(1)$	$O(n)$, with shifting
Rehashing / resizing	Rehashing / resizing	Resizing includes rehashing
Cache /memory	weak for large tables	Strong
Main strength	Worst case lookup	Cache locality
Main weakness	Insertion cycles and rehashing	Shifts
Hashing functions	2	1

All together

Method	Insert	Lookup	Delete	Mem
Cuckoo Hashing	$O(1)$, $O(n)$	$O(1)$	$O(1)$	$O(n)$
Chained Hashing	$O(1)$	$O(1)$, $O(n)$	$O(1)$, $O(n)$	$O(n)$
Linear Probing	$O(1)$, $O(n)$	$O(1)$, $O(n)$	$O(1)$, $O(n)$	$O(n)$
Two-Way Chaining	$O(1)$	$O(1)$, $O(\log \log n)$	$O(1)$, $O(\log \log n)$	$O(n)$
Robin Hood Hashing	$O(1)$, $O(n)$	$O(1)$, $O(n)$	$O(1)$, $O(n)$	$O(n)$

Color code:

Green: Average

Red: Worst

Orange: Amortized

Cuckoo Hashing | Dataset

- [23] G. Marsaglia, The Marsaglia random number CDROM including the diehard battery of tests of randomness, <http://stat.fsu.edu/pub/diehard/>.

The CDROM I will soon be distributing contains some 60 ten-meg binary files of random numbers for use in serious Monte Carlo calculations as well as for input to DIEHARD. Most of those files combine several of the best deterministic generators with white noise and black noise, (the latter from a CD of rap music). Some of the files contain only random bits from physical devices. They fail many of the tests in DIEHARD.

No success to find the data in the mentioned reference

Cuckoo Hashing | Dataset

Dataset:

- re-creation of George Marsaglia's original random-number CD-ROM.
- 60 binary files, each about 10 MB,
- bit.01, bit.02, ..., bit.60
- with a total of about 4.8 billion random bits
- For the hashing experiments, the binary data is read as a stream of 32-bit unsigned integer keys.
- 32-bit unsigned integer keys: $\frac{4.8 \text{ billion bits}}{32 \text{ bits/key}} = 150 \text{ million keys}$

Found on github <https://github.com/jeffThompson/DiehardCDROM/tree/master>

THE MARSAGLIA
RANDOM NUMBER CDROM
including the
DIEHARD BATTERY OF TESTS
OF RANDOMNESS

Research
Sponsored by
THE NATIONAL
SCIENCE
FOUNDATION
Grants
DMS-8807976
DMS-9206972

DEPARTMENT
OF STATISTICS
and
SUPERCOMPUTER
COMPUTATIONS
RESEARCH
INSTITUTE

© 1995
George Marsaglia
All rights reserved

Professor
George Marsaglia
geo@stat.fsu.edu



Experiment setup | Cuckoo Hashing & Robin Hood

Exp.	(n) keys	Table size (4n)	# Operations (3n * 4)	log(n)	Rep.	Max loop	Algorithm
1	256	1,024	768	8	10	64	Cuckoo, robin hood
2	512	2,048	1,536	9			
3	1,024	4,096	3,072	10			
4	2,048	8,192	6,144	11			
5	4,096	16,384	12,288	12			
6	8,192	32,768	24,576	13			
7	16,384	65,536	49,152	14			
8	32,768	131,072	98,304	15			
9	65,536	262,144	196,608	16			
10	131,072	524,288	393,216	17			
11	262,144	1,048,576	786,432	18			
12	524,288	2,097,152	1,572,864	19			
13	1,048,576	4,194,304	3,145,728	20			
14	2,097,152	8,388,608	6,291,456	21			
15	4,194,304	16,777,216	12,582,912	22			

(n) keys: pre operations (not timed)

Table size: $4n$, $\alpha = 0.25$. In CH, size of each table = Table size / 2 (not timed)

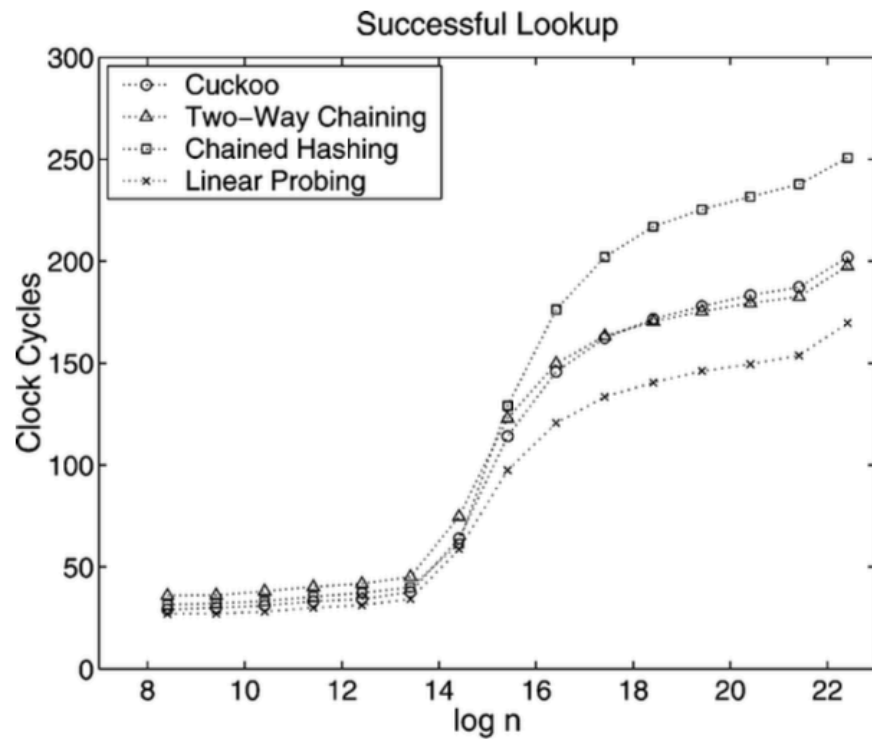
Each operation includes 4 functions: unsuccessful lookup, successful lookup, delete, insert.

Repetition (Rep.): each test is repeated 10 times, then results are averaged.

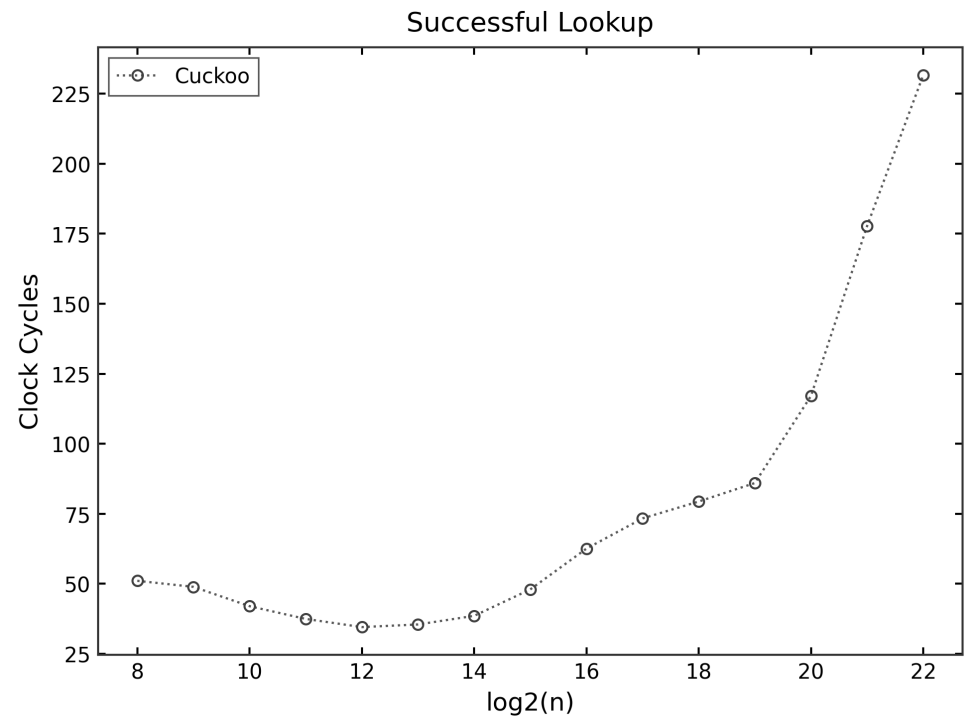
Cuckoo Hashing | Experimental Results

Successful Lookup

Paper results



Experimental results

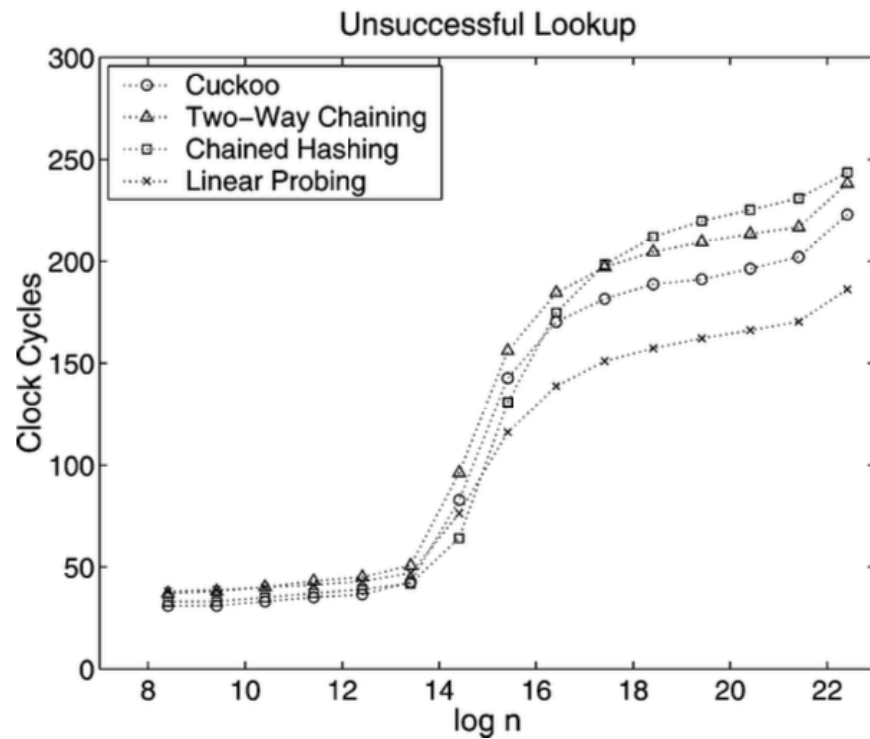


Successful lookup operations: $(3n) \times 10$

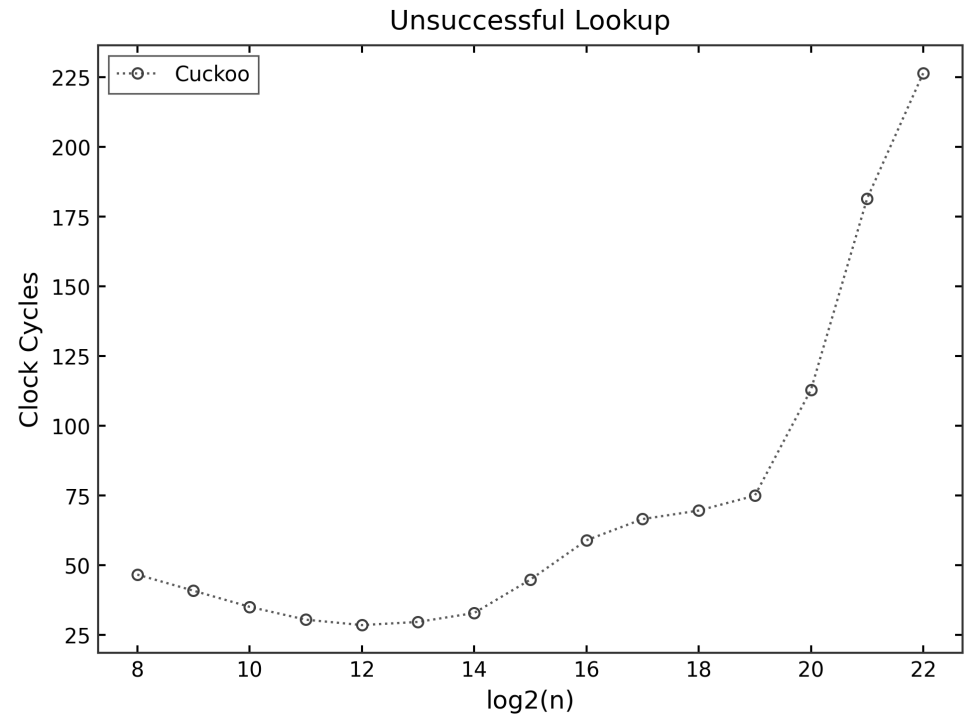
Cuckoo Hashing | Experimental Results

Unsuccessful Lookup

Paper results



Experimental results

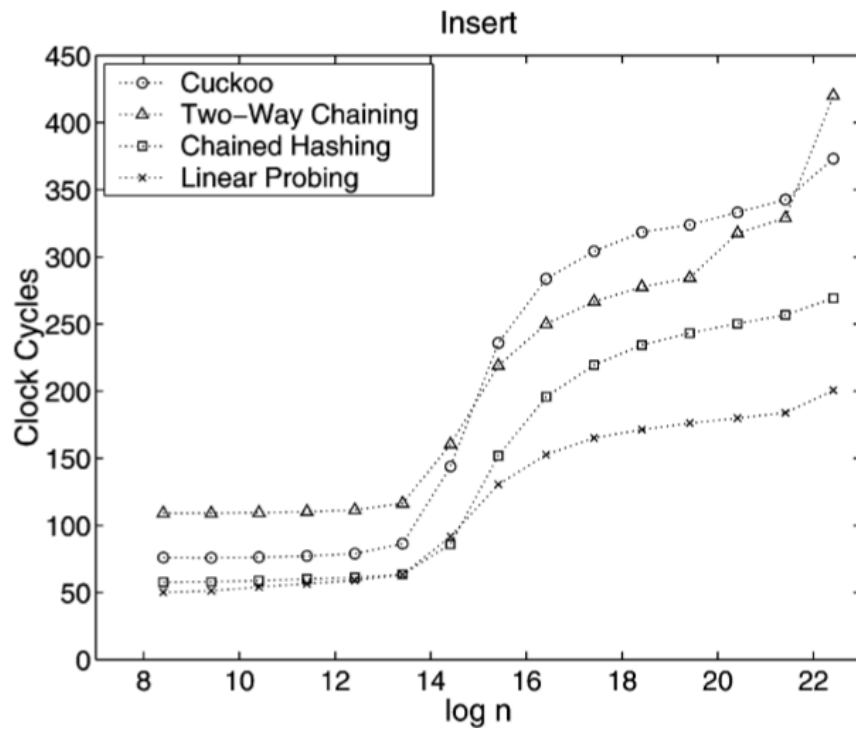


Unsuccessful lookup operations: $(3n) \times 10$

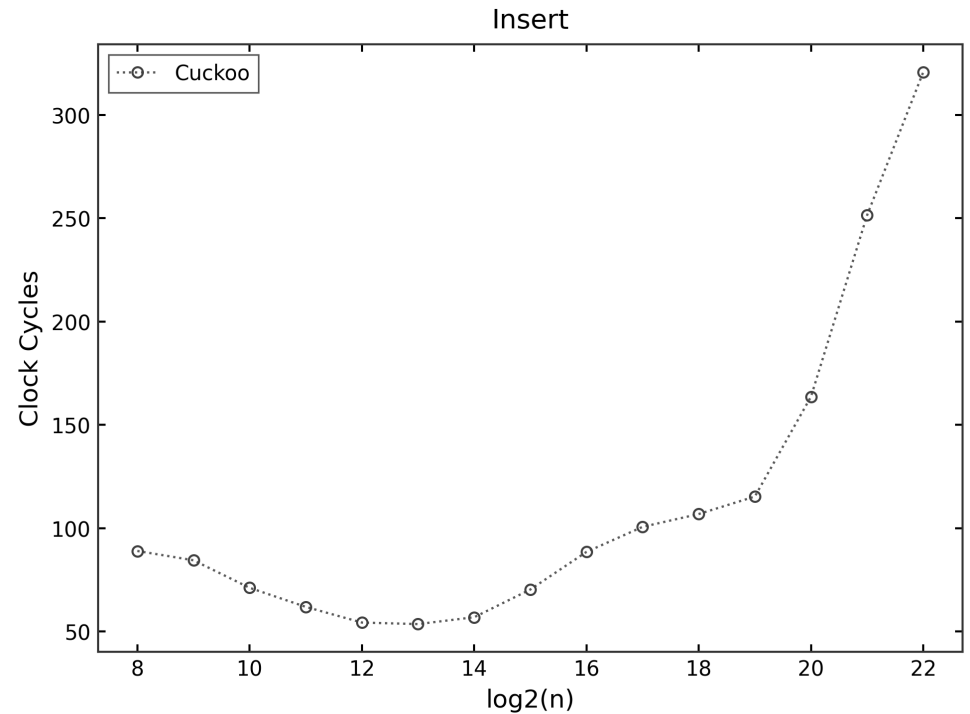
Cuckoo Hashing | Experimental Results

Insertion:

Paper results



Experimental results

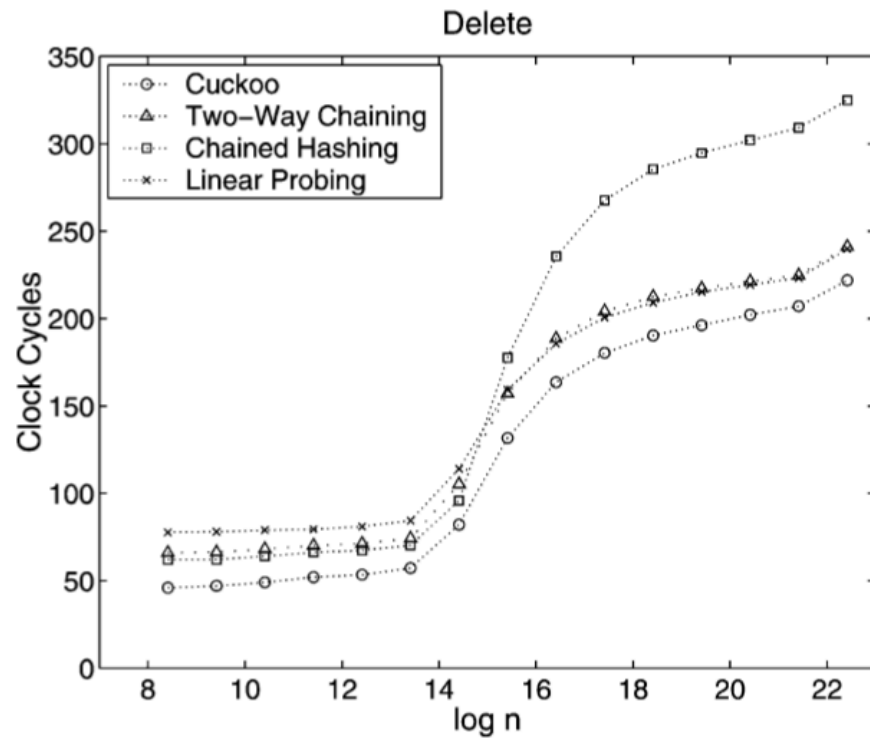


Insert operations: $(3n) \times 10$

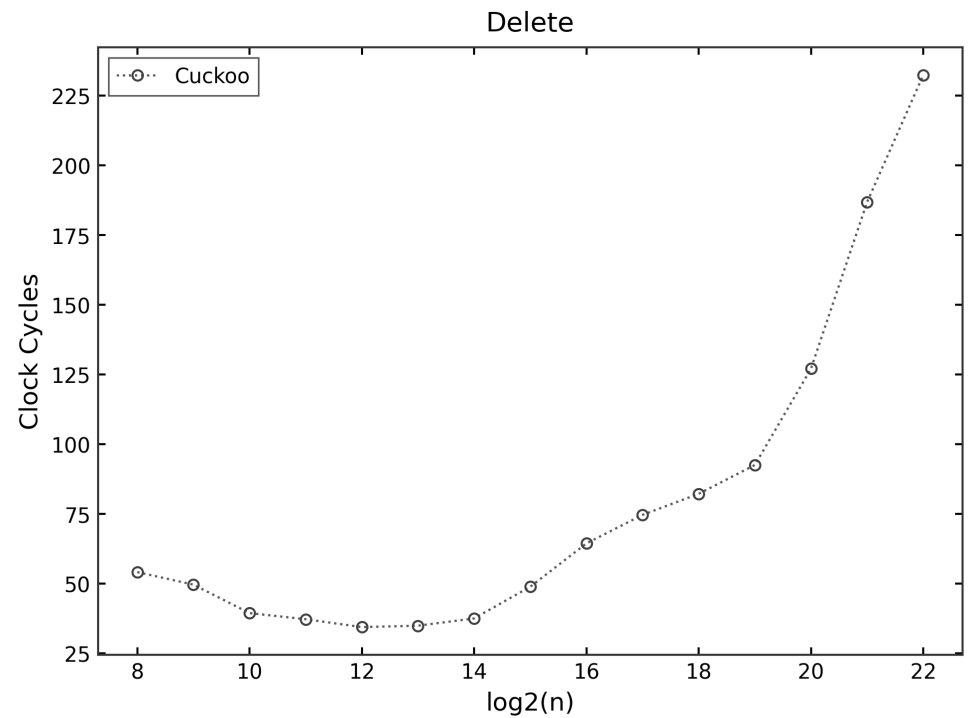
Cuckoo Hashing | Experimental Results

Delete:

Paper results



Experimental results

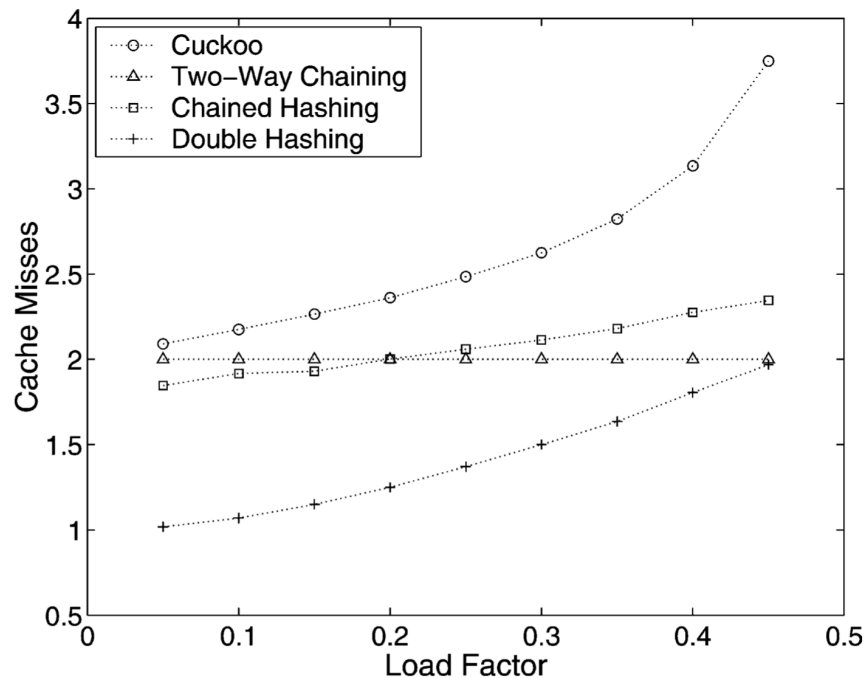


Delete operations: $(3n) \times 10$

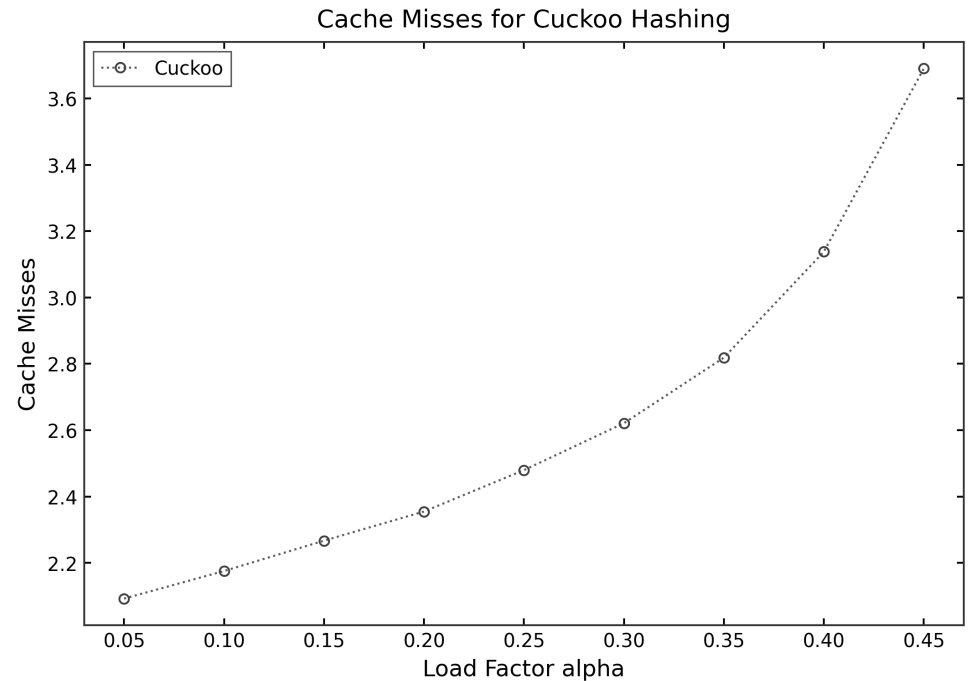
Cuckoo Hashing | Experimental Results

Random memory access:

Paper results



Experimental results

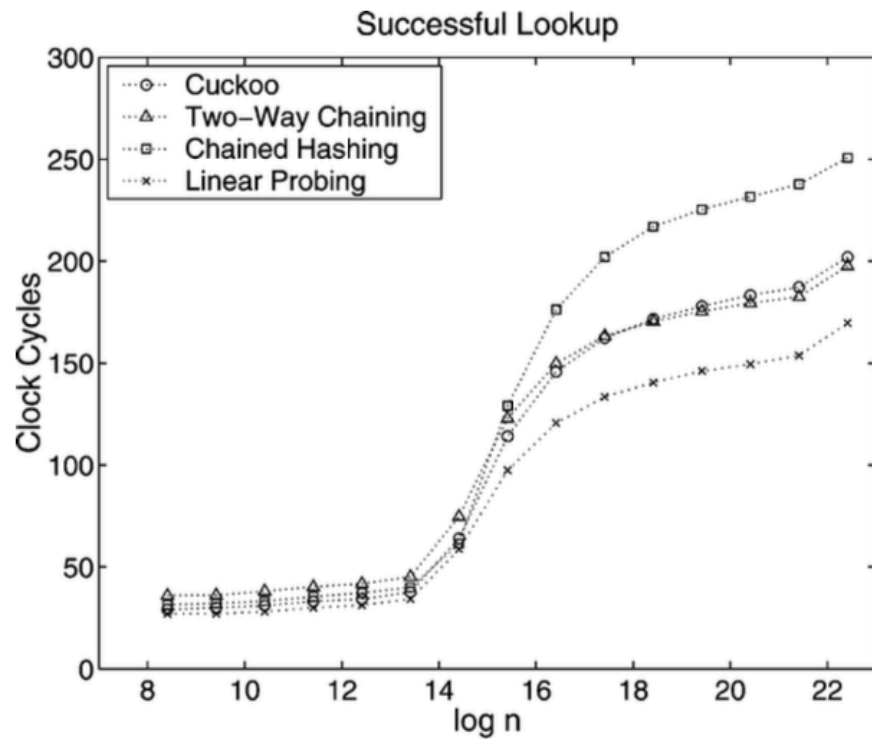


Average random memory cell accesses per insertion. Measurements are taken in equilibrium after 10^5 insertions and deletions, using tables of size 2^{15} across different load factors.

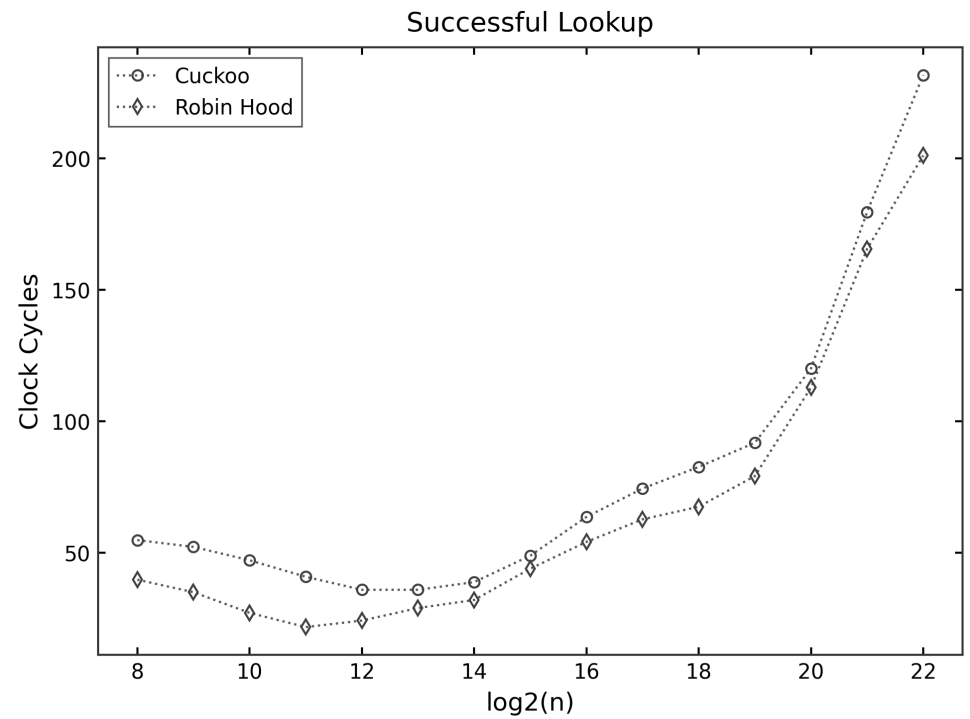
Cuckoo vs Robin Hood | Experimental Results

Successful Lookup

Paper results



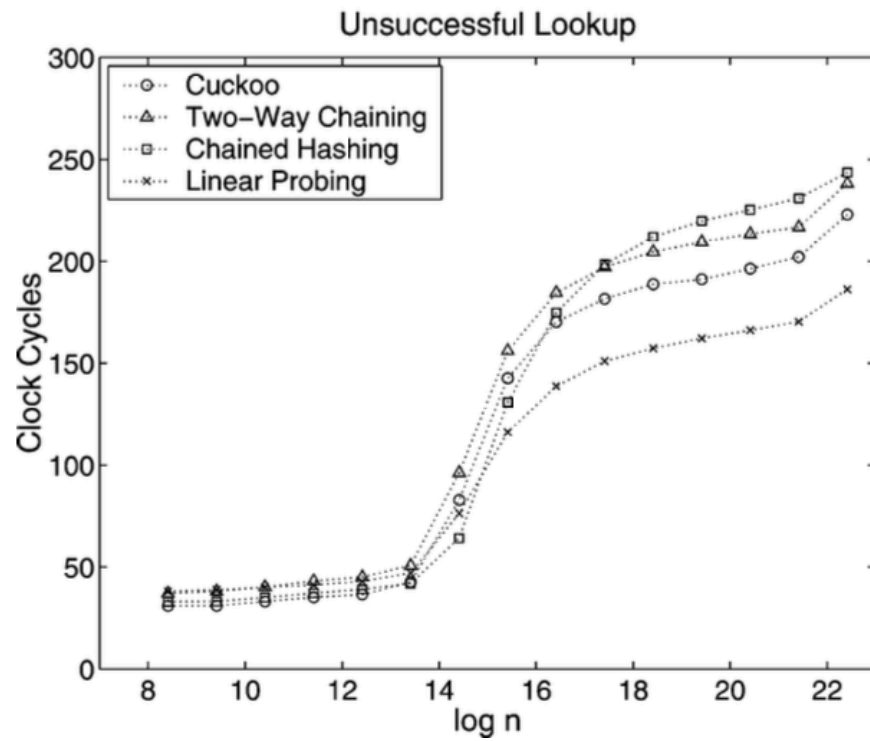
Experimental results



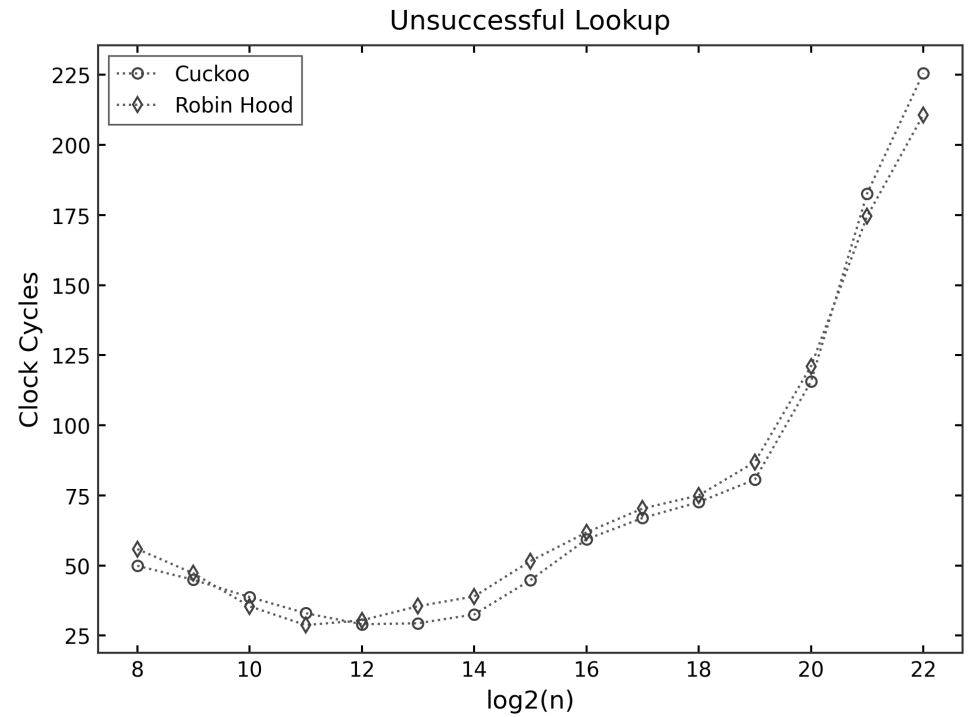
Cuckoo vs Robin Hood | Experimental Results

Unsuccessful Lookup

Paper results



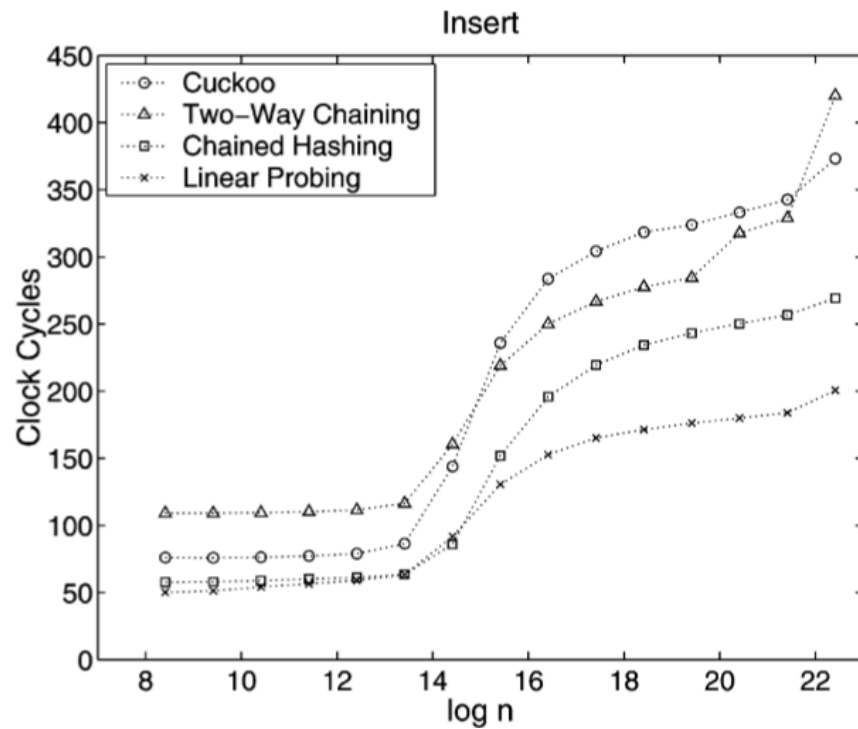
Experimental results



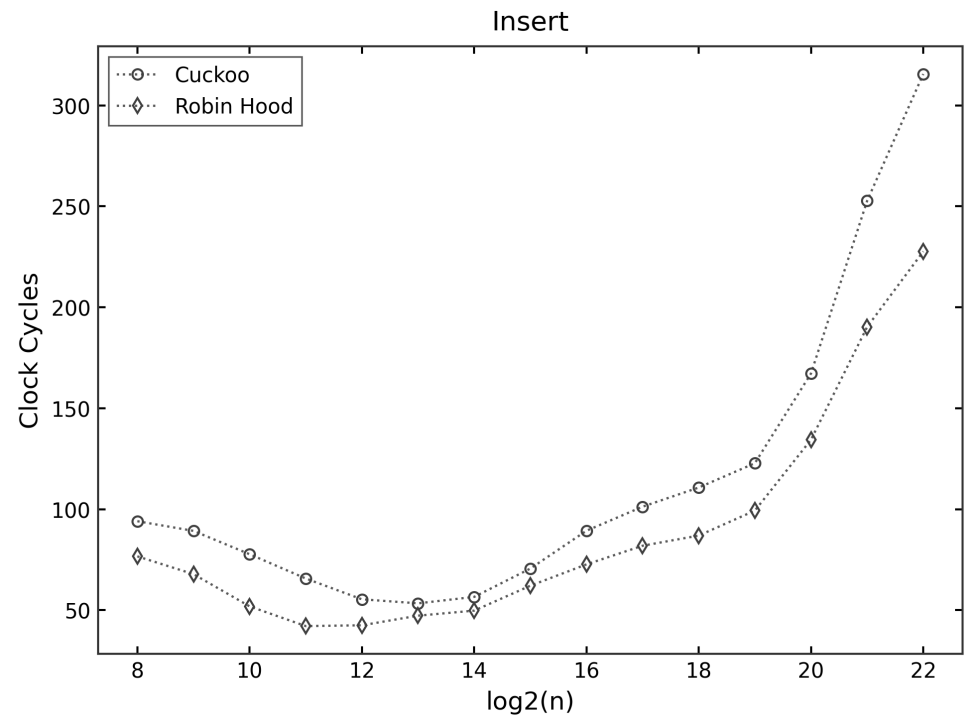
Cuckoo vs Robin Hood | Experimental Results

Insertion:

Paper results



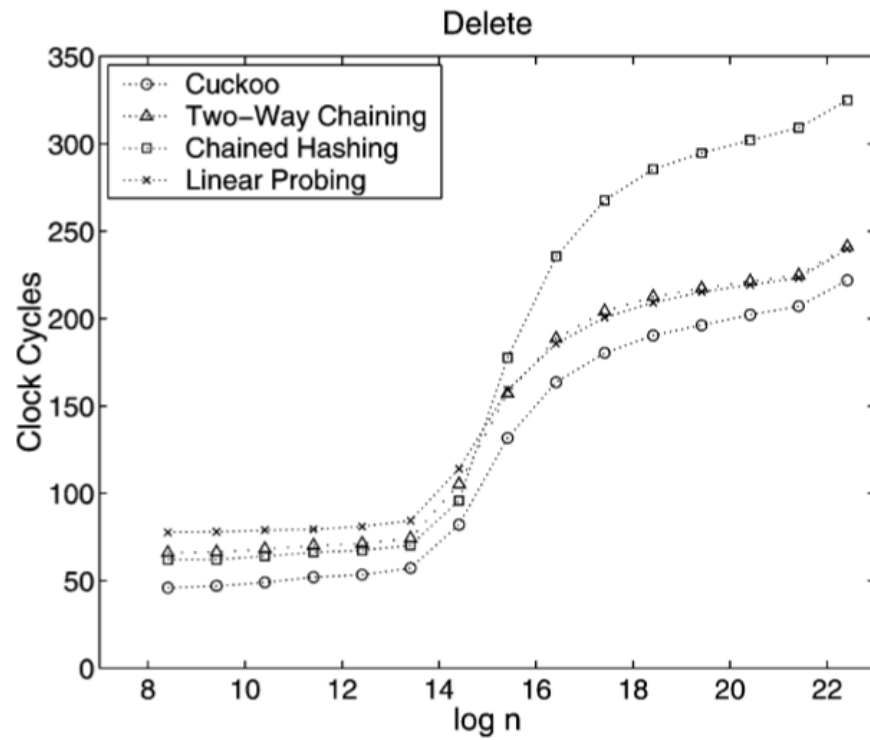
Experimental results



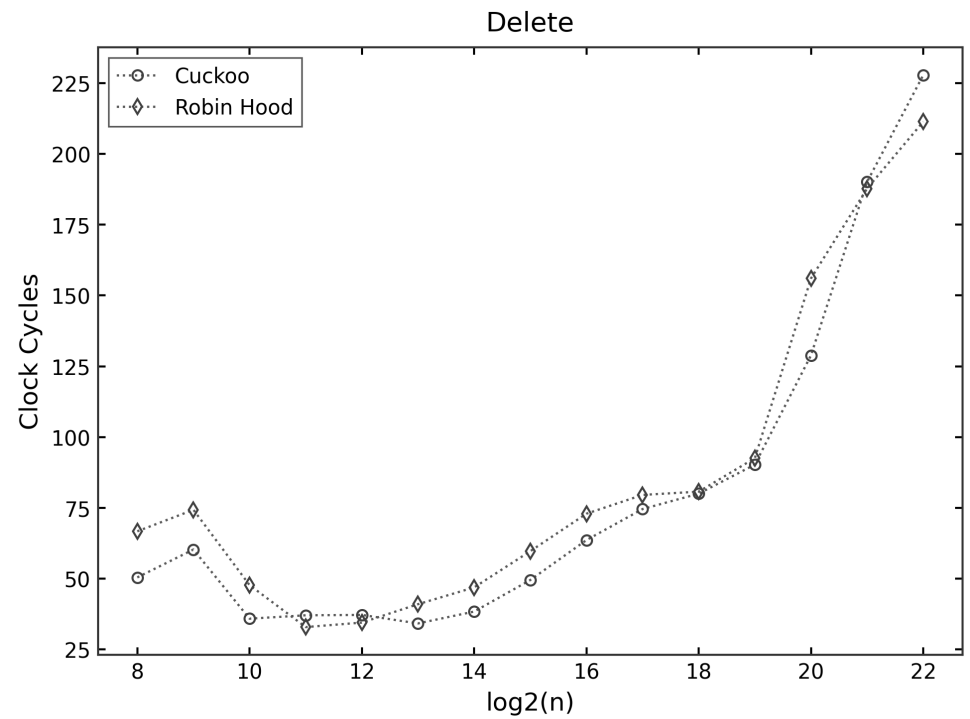
Cuckoo vs Robin Hood | Experimental Results

Delete:

Paper results



Experimental results





Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

Thanks

Mustafa Alzubi

mus20239002@std.psut.edu.jo